

HỌC VIỆN CÔNG NGHỆ BƯU CHÍNH VIỄN THÔNG
KHOA CÔNG NGHỆ THÔNG TIN 1

BÁO CÁO THỰC TẬP TỐT NGHIỆP
XÂY DỰNG WEBSITE ĐỌC TRUYỆN
TRỰC TUYẾN

Sử dụng React và Java Spring Boot

Giảng viên hướng dẫn: TS. Vũ Trọng Sinh

Sinh viên thực hiện: Lâm Thành Đức

Mã sinh viên: B22DCCN227

Tên sản phẩm: MangaVault

HÀ NỘI – 2026

LỜI CẢM ƠN

Trong quá trình thực hiện đề tài “Xây dựng website đọc truyện trực tuyến sử dụng React và Java Spring Boot”, em đã nhận được sự hướng dẫn, hỗ trợ và góp ý từ giảng viên, các thầy cô trong khoa cùng những nguồn tài liệu chuyên ngành. Em xin gửi lời cảm ơn chân thành tới TS. Vũ Trọng Sinh đã định hướng đề tài, giúp em xác định phạm vi phù hợp và nhắc nhở về cách tiếp cận một sản phẩm phần mềm có cấu trúc.

Em cũng xin cảm ơn các thầy cô Khoa Công nghệ Thông tin 1, Học viện Công nghệ Bưu chính Viễn thông đã trang bị nền tảng về lập trình, cơ sở dữ liệu, mạng máy tính, công nghệ phần mềm và phát triển ứng dụng web. Những kiến thức đó là điểm xuất phát để em có thể tự nghiên cứu Spring Boot, React, JWT, Docker và quy trình triển khai sản phẩm lên môi trường thật.

Quá trình thực tập giúp em nhận ra rằng một ứng dụng chạy được trên máy cá nhân mới chỉ là bước đầu. Để trở thành sản phẩm có thể sử dụng, hệ thống còn cần quản lý yêu cầu, phân quyền ở backend, bảo toàn dữ liệu, xử lý lỗi nhất quán, kiểm thử, cấu hình môi trường, triển khai và theo dõi các vấn đề phát sinh. Đây là trải nghiệm có ý nghĩa lớn đối với định hướng nghề nghiệp của em.

MỤC LỤC

MỞ ĐẦU	6
1. Lý do lựa chọn đề tài	6
2. Mục tiêu	6
3. Phương pháp thực hiện	6
CHƯƠNG 1. TỔNG QUAN VÀ KHẢO SÁT HỆ THỐNG	7
1.1. Bối cảnh và bài toán	7
1.2. Đối tượng sử dụng	7
1.3. Khảo sát giải pháp tương tự	7
1.4. Phạm vi sản phẩm	7
CHƯƠNG 2. PHÂN TÍCH YÊU CẦU	8
2.1. Yêu cầu chức năng	8
2.2. Yêu cầu phi chức năng	8
2.3. Các luồng nghiệp vụ tiêu biểu	9
2.3.1. Đăng nhập	9
2.3.2. Đọc và lưu tiến độ	9
2.3.3. Kiểm duyệt bình luận	9
CHƯƠNG 3. PHÂN TÍCH VÀ THIẾT KẾ HỆ THỐNG	10
3.1. Kiến trúc tổng thể	10
3.2. Công nghệ sử dụng	11
3.3. Thiết kế dữ liệu	11
3.4. Thiết kế API	12
3.5. Thiết kế bảo mật	13
3.5.1. Ma trận kiểm soát truy cập	13
CHƯƠNG 4. TRIỂN KHAI SẢN PHẨM	14
4.1. Tổ chức mã nguồn	14

4.2. Backend	14
4.2.1. Story và tìm kiếm	14
4.2.2. Chapter và điều hướng	14
4.2.3. Rating và dữ liệu tổng hợp	15
4.2.4. Xóa dữ liệu	15
4.3. Frontend.....	15
4.4. Các chức năng quản trị	15
CHƯƠNG 5. KIỂM THỬ, TRIỂN KHAI VÀ KẾT QUẢ	16
5.1. Chiến lược kiểm thử	16
5.2. Triển khai.....	16
5.3. Đánh giá kết quả.....	17
CHƯƠNG 6. KINH NGHIỆM THỰC TẬP VÀ NHỮNG ĐIỀU HỌC ĐƯỢC	18
6.1. Khác biệt với việc học làm web trên trường	18
6.2. Bài học kỹ thuật cụ thể	18
Kiến trúc phân lớp	18
DTO và API contract.....	18
JWT và Spring Security	19
CORS.....	19
Transaction	19
ORM và hiệu năng.....	19
Dữ liệu tổng hợp.....	19
Kiểm thử	19
Frontend state	19
Deployment	19
6.3. Những vấn đề thực tế đã gặp và cách giải quyết.....	20
6.4. Kỹ năng làm việc và tư duy nghề nghiệp	21
6.5. Tự đánh giá sự tiến bộ	21

CHƯƠNG 7. HẠN CHẾ VÀ HƯỚNG PHÁT TRIỂN	22
7.1. Hạn chế hiện tại	22
7.2. Hướng phát triển.....	22
7.3. Giá trị có thể tái sử dụng	Error! Bookmark not defined.
KẾT LUẬN	23
TÀI LIỆU THAM KHẢO	Error! Bookmark not defined.
PHỤ LỤC A. HƯỚNG DẪN CHẠY DỰ ÁN.....	Error! Bookmark not defined.
A.1. Yêu cầu môi trường	Error! Bookmark not defined.
A.2. Các bước	Error! Bookmark not defined.
PHỤ LỤC B. CẤU HÌNH MÔI TRƯỜNG	Error! Bookmark not defined.
PHỤ LỤC C. THÔNG TIN SẢN PHẨM	Error! Bookmark not defined.

Ghi chú: Mục lục là trường tự động của Microsoft Word. Khi mở tài liệu, chọn toàn bộ (Ctrl+A) và nhấn F9 để cập nhật số trang nếu cần.

MỞ ĐẦU

1. Lý do lựa chọn đề tài

Nhu cầu tiếp cận nội dung số trên trình duyệt và thiết bị di động ngày càng phổ biến. Website đọc truyện chữ là bài toán đủ gần gũi để đánh giá trải nghiệm người dùng, đồng thời có chiều sâu kỹ thuật ở quản lý nội dung, xác thực, phân quyền, tìm kiếm, lịch sử đọc, tương tác cộng đồng và quản trị hệ thống. Vì vậy, đề tài phù hợp để vận dụng kiến thức frontend, backend và cơ sở dữ liệu trong một sản phẩm thống nhất.

MangaVault được xây dựng theo kiến trúc client-server tách biệt. React đảm nhiệm giao diện; Spring Boot cung cấp REST API và xử lý nghiệp vụ; MySQL lưu trữ dữ liệu. Phạm vi ban đầu tập trung vào đọc truyện và quản lý truyện/chương, sau đó mở rộng thành sản phẩm đầy đủ cho ba nhóm GUEST, USER và ADMIN.

2. Mục tiêu

- Xây dựng website đọc truyện chữ có thể hoạt động trên trình duyệt và nhiều kích thước màn hình.
- Thiết kế REST API có cấu trúc, sử dụng DTO và xử lý lỗi tập trung.
- Áp dụng Spring Security, BCrypt và JWT để xác thực, phân quyền.
- Triển khai đầy đủ chức năng đọc, theo dõi, lịch sử, bình luận, đánh giá và quản trị.
- Đóng gói, kiểm thử và triển khai sản phẩm lên môi trường thật.
- Rút ra bài học về quy trình phát triển phần mềm thực tế so với bài tập web trong nhà trường.

3. Phương pháp thực hiện

1. Khảo sát sản phẩm tương tự và xác định yêu cầu F01–F22.
2. Thiết kế domain, kiến trúc phân lớp, API và mô hình dữ liệu.
3. Phát triển theo từng giai đoạn nhỏ, kiểm tra luồng end-to-end sau mỗi giai đoạn.
4. Dùng Git/GitHub quản lý phiên bản, Postman kiểm tra API, H2 cho integration test.
5. Đóng gói backend bằng Docker và triển khai frontend/backend/database trên các nền tảng cloud.

CHƯƠNG 1. TỔNG QUAN VÀ KHẢO SÁT HỆ THỐNG

1.1. Bối cảnh và bài toán

Người đọc trực tuyến cần một hệ thống giúp khám phá nội dung nhanh, đọc tập trung và ghi nhớ tiến độ. Người quản trị cần công cụ quản lý truyện, chương, thể loại, người dùng và nội dung tương tác. Nếu chỉ xây một trang hiển thị văn bản, hệ thống sẽ không giải quyết được các vấn đề về tổ chức dữ liệu, quyền truy cập và khả năng vận hành lâu dài.

1.2. Đối tượng sử dụng

Đối tượng	Nhu cầu chính	Ranh giới quyền
GUEST	Xem, tìm kiếm, lọc, đọc truyện; xem bình luận và điểm đánh giá	Không thực hiện thao tác cá nhân hoặc quản trị
USER	Hồ sơ, theo dõi, lịch sử, đọc tiếp, bình luận, báo cáo, đánh giá	Chỉ sửa/xóa nội dung thuộc sở hữu
ADMIN	Quản trị truyện, chương, thể loại, user, bình luận, thống kê	Backend yêu cầu ROLE_ADMIN cho /api/admin/**

1.3. Khảo sát giải pháp tương tự

Wattpad nổi bật ở tương tác cộng đồng, thư viện cá nhân và mô hình người dùng vừa đọc vừa sáng tác. Royal Road tập trung vào web novel, có lọc chi tiết, theo dõi chương và công cụ quản lý nội dung. Từ khảo sát, MangaVault lựa chọn ba trục cốt lõi: khám phá nội dung, trải nghiệm đọc và tương tác/quản trị. Các chức năng mạng xã hội tác giả, kiếm tiền và đề xuất bằng trí tuệ nhân tạo nằm ngoài phạm vi phiên bản hiện tại.

1.4. Phạm vi sản phẩm

Sản phẩm quản lý truyện chữ; mỗi truyện có thông tin mô tả, ảnh bìa, trạng thái, độ hiển thị, nhiều thể loại và danh sách chương. Mỗi chương chứa nội dung văn bản. Hệ thống không tối ưu cho truyện tranh dung lượng lớn, âm thanh hoặc video. Tác giả hiện được lưu như thuộc tính của truyện thay vì là tài khoản xuất bản nội dung.

CHƯƠNG 2. PHÂN TÍCH YÊU CẦU

2.1. Yêu cầu chức năng

Mã	Nhóm chức năng	Hiện trạng triển khai
F01	Trang chủ	Các khu vực mới cập nhật, nổi bật, xem nhiều; tìm kiếm và thể loại
F02	Danh sách truyện	Phân trang, hiển thị thông tin tóm tắt
F03	Tìm kiếm	Theo tên, tác giả hoặc thể loại
F04	Lọc/sắp xếp	Thể loại, trạng thái; latest/updated/views/rating/title
F05–F07	Đọc truyện	Chi tiết, mục lục, nội dung, điều hướng chương
F08–F10	Tài khoản	Đăng ký; login username/email; hồ sơ và đổi mật khẩu
F11–F13	Thư viện cá nhân	Theo dõi, lịch sử, tiếp tục đọc
F14–F15	Tương tác	Bình luận/báo cáo và đánh giá sao
F16–F18	Quản trị nội dung	CRUD truyện, chương, thể loại; publish/reorder
F19–F21	Quản trị hệ thống	User, moderation, thống kê
F22	Tùy chọn đọc	Cỡ chữ, giãn dòng, giao diện sáng/tối

2.2. Yêu cầu phi chức năng

Bảo mật: Mật khẩu BCrypt; JWT Bearer; phân quyền backend; không trả password hash; CORS theo origin; cấu hình nhảy cảm qua biến môi trường.

Hiệu năng: Phân trang; summary DTO không chứa chapter content; index; lazy loading; batch fetch; tránh query theo từng phần tử ở danh sách theo dõi.

Toàn vẹn: Unique username/email/slug; chapterNumber duy nhất trong story; một rating/follow/progress mỗi user–story; transaction khi xóa hoặc cập nhật liên quan nhiều bảng.

Bảo trì: Controller–Service–Repository; constructor injection; API call frontend tập trung trong services; lỗi và response thống nhất.

Trải nghiệm: Responsive; chế độ sáng/tối; tùy chỉnh trình đọc; trạng thái loading/error; điều hướng rõ ràng.

2.3. Các luồng nghiệp vụ tiêu biểu

2.3.1. Đăng nhập

Người dùng nhập username hoặc email và mật khẩu. Backend tìm tài khoản không phân biệt hoa thường, so khớp BCrypt, kiểm tra trạng thái tài khoản rồi phát JWT HS256 chứa uid và role. Frontend lưu token, Axios interceptor gắn Authorization vào yêu cầu sau. Khi nhận 401, phiên cũ được xóa để giao diện trở lại trạng thái guest.

2.3.2. Đọc và lưu tiến độ

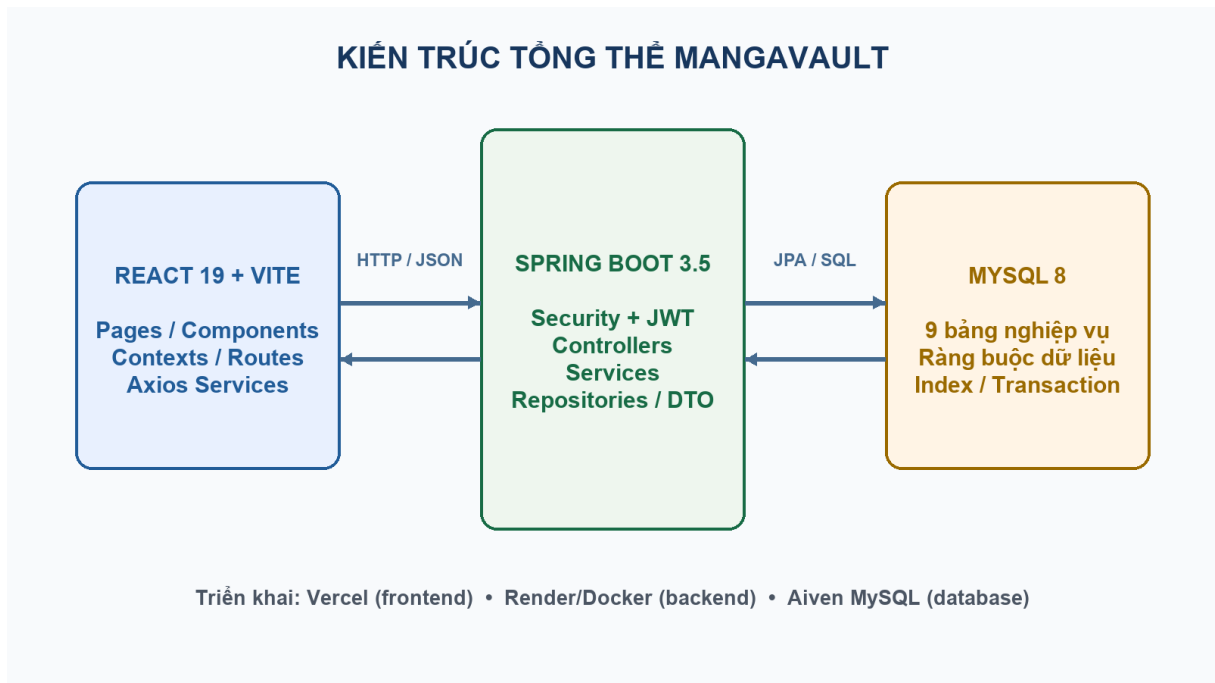
API đọc chương chỉ cho phép chapter đã publish thuộc story PUBLIC. Service tìm chương trước/sau gần nhất thay vì giả định số chương liên tục. Lướt xem được tăng bằng bulk update để không làm thay đổi updatedAt của truyện. Nếu đã đăng nhập, frontend gửi yêu cầu upsert tiến độ đọc theo cặp user–story.

2.3.3. Kiểm duyệt bình luận

User có thể báo cáo một bình luận một lần. Admin tìm kiếm toàn bộ bình luận, lọc báo cáo hoặc trạng thái ẩn, sau đó bỏ báo cáo, ẩn/khôi phục, xóa hoặc khóa tác giả. Bình luận ẩn được giữ lại làm dấu vết quản trị nhưng không xuất hiện trên API công khai.

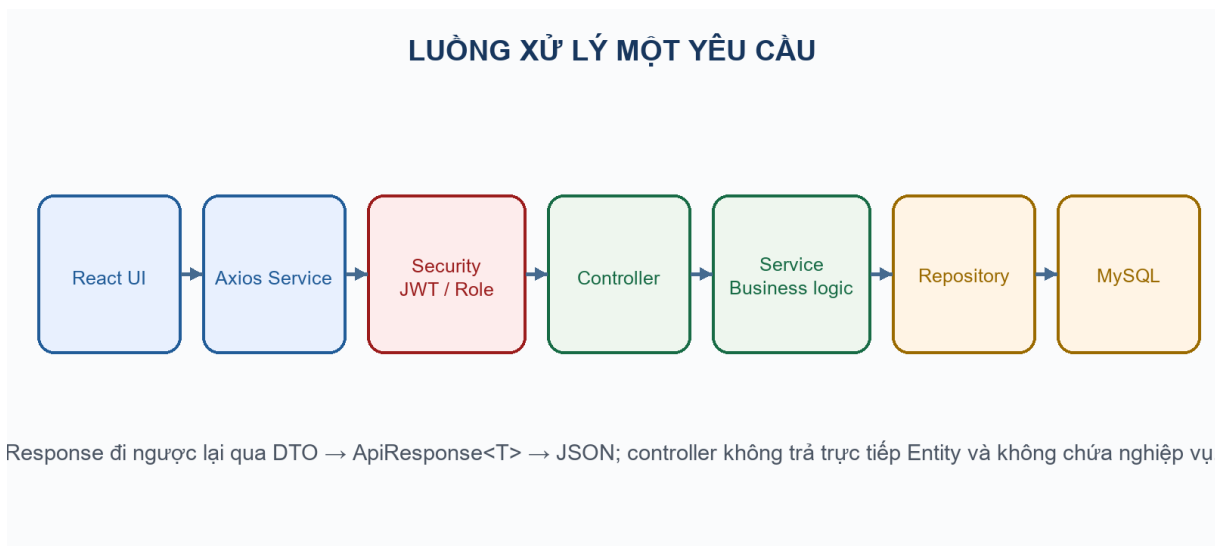
CHƯƠNG 3. PHÂN TÍCH VÀ THIẾT KẾ HỆ THỐNG

3.1. Kiến trúc tổng thể



Hình 3.1. Kiến trúc tổng thể của MangaVault

Kiến trúc tách frontend và backend giúp hai phần phát triển, triển khai và mở rộng độc lập. Frontend không truy cập trực tiếp database. Mọi dữ liệu đi qua REST API và ranh giới Spring Security. Backend phân lớp để controller chỉ tiếp nhận request/validation cơ bản, service xử lý nghiệp vụ và repository làm việc với dữ liệu.

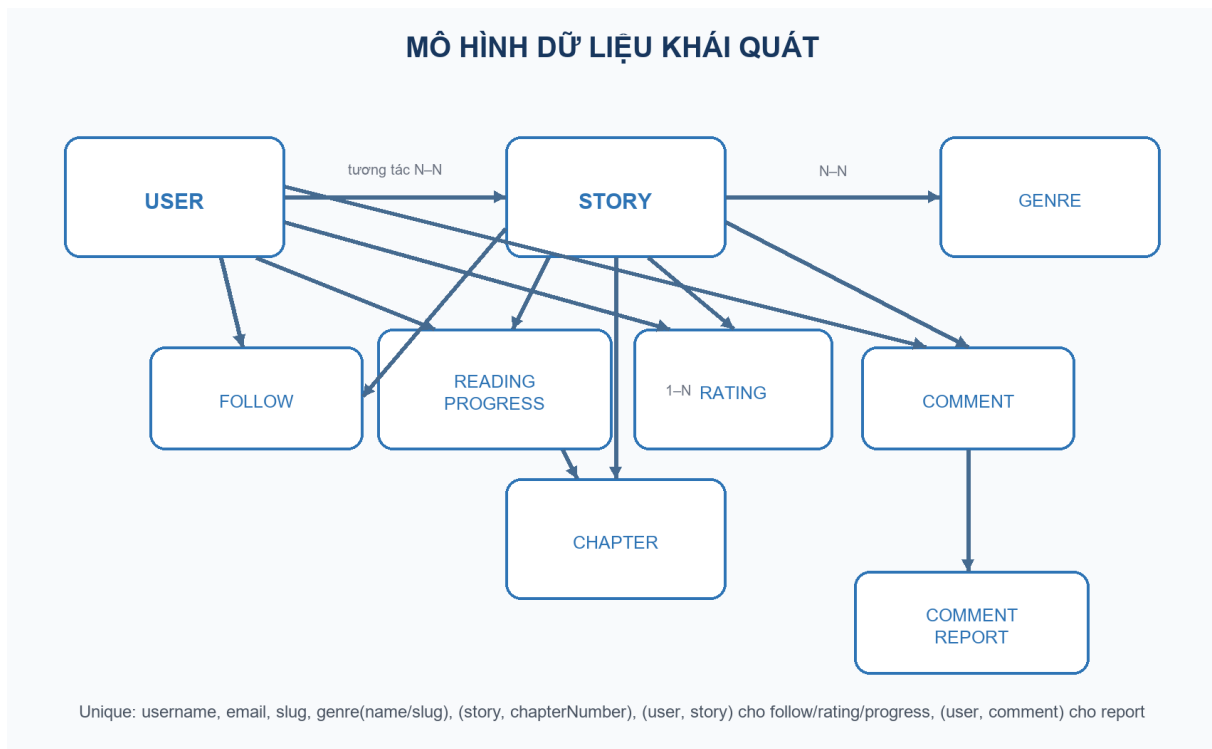


Hình 3.2. Luồng xử lý chuẩn từ giao diện đến cơ sở dữ liệu

3.2. Công nghệ sử dụng

Thành phần	Công nghệ	Vai trò
Frontend	React 19, Vite 7, React Router 7, Axios	SPA, routing, context, gọi REST API
Backend	Java 21, Spring Boot 3.5	Web API, validation, service, cấu hình
Security	Spring Security, OAuth2 Resource Server, JWT HS256, BCrypt	Xác thực và phân quyền stateless
Persistence	Spring Data JPA, Hibernate, MySQL 8	ORM, repository và lưu dữ liệu
Test	JUnit 5, MockMvc, Spring Security Test, H2	Integration test không phụ thuộc MySQL
Triển khai	Docker, Render, Vercel, Aiven	Đóng gói và vận hành cloud
Công cụ	Git/GitHub, Postman	Phiên bản mã nguồn và kiểm tra API

3.3. Thiết kế dữ liệu



Hình 3.3. Quan hệ giữa các entity chính

Entity	Trách nhiệm dữ liệu
User	Tài khoản, BCrypt password, role, status, hồ sơ
Story	Tên, slug, tác giả, mô tả, bìa, trạng thái, visibility, counters
Genre	Tên, slug, mô tả
Chapter	Story, thứ tự, tiêu đề, content, published
Follow	Quan hệ theo dõi user-story
ReadingProgress	Chương và thời điểm đọc gần nhất
Rating	Điểm 1-5; duy nhất theo user-story
Comment	Nội dung, reportCount, hidden
CommentReport	Người báo cáo, lý do; duy nhất theo user-comment

3.4. Thiết kế API

Quyền	Endpoint tiêu biểu	Mục đích
Public	GET /api/stories	Danh sách, tìm kiếm, lọc, phân trang
Public	GET /api/stories/{id}	Chi tiết story PUBLIC
Public	GET /api/chapters/{id}	Nội dung chương + prev/next
Auth	POST /api/auth/register login	Tạo USER; phát JWT
USER	/api/me/**	Hồ sơ, mật khẩu, follow, history
USER	/api/stories/{id}/comments rating	Tương tác nội dung
ADMIN	/api/admin/stories/**	CRUD truyện/chương
ADMIN	/api/admin/genres users	Thẻ loại và tài khoản
ADMIN	/api/admin/comments stats	Moderation và dashboard

Tất cả response nghiệp vụ dùng envelope `ApiResponse<T>` với code, result và message. Danh sách phân trang dùng `PageResponse<T>`. Lỗi nghiệp vụ được mô hình hóa bằng `ErrorCode` chứa mã ứng dụng, thông điệp và HTTP status; `GlobalExceptionHandler` trả định dạng nhất quán cho frontend.

3.5. Thiết kế bảo mật

- JWT được gửi qua Authorization: Bearer <token>; backend không dùng session.
- Claim role được chuyển thành ROLE_ADMIN/ROLE_USER để Spring Security kiểm tra URL.
- Route public được khai báo rõ; /api/admin/** yêu cầu ADMIN; API cá nhân yêu cầu authenticated.
- Mật khẩu chỉ lưu BCrypt hash; login trả cùng một lỗi cho username không tồn tại và mật khẩu sai.
- Dữ liệu PRIVATE/unpublished được trả như 404 để tránh tiết lộ sự tồn tại.
- Quyền sở hữu bình luận được kiểm tra ở service; việc ấn nút trên React chỉ phục vụ UX.

3.5.1. Ma trận kiểm soát truy cập

Nhóm API	Đối tượng	Cơ chế bảo vệ
Danh sách, chi tiết truyện; chương; thể loại; bình luận và điểm đánh giá công khai	GUEST, USER, ADMIN	Cho phép công khai; dữ liệu trả qua DTO và phân trang khi cần.
Hồ sơ, theo dõi, lịch sử đọc, đánh giá và bình luận cá nhân	USER, ADMIN	Yêu cầu JWT; backend kiểm tra người dùng hiện tại và quyền sở hữu tài nguyên.
/api/admin/**	ADMIN	Yêu cầu JWT và role ADMIN; việc ấn nút ở frontend không thay thế kiểm tra backend.

Lỗi xác thực được xử lý ngay trong chuỗi Spring Security: thiếu hoặc sai token trả về 401; đã đăng nhập nhưng không đủ quyền trả về 403. Cách xử lý này giữ response lỗi nhất quán cả khi request chưa đi tới controller.

JWT secret, thông tin kết nối cơ sở dữ liệu và cấu hình triển khai không được ghi trực tiếp vào mã nguồn. Chúng được truyền qua biến môi trường; CORS chỉ chấp nhận origin frontend hợp lệ ở từng môi trường.

CHƯƠNG 4. TRIỂN KHAI SẢN PHẨM

4.1. Tổ chức mã nguồn

Repository hiện có 87 file Java thuộc mã nguồn chính, 3 file kiểm thử, 35 file JSX và 9 file JavaScript trong frontend. Backend nằm ở root theo cấu trúc Maven; frontend nằm trong thư mục frontend/. Cách tổ chức giữ luồng build hiện tại ổn định và tránh di chuyển dự án chỉ vì mục đích hình thức.

Thư mục	Nội dung
controller/	REST endpoint, request param/body, response envelope
service/	Toàn bộ nghiệp vụ, transaction, mapping DTO
repository/	Spring Data JPA, JPQL, aggregate/bulk update
entity/	9 entity và ràng buộc quan hệ
dto/	Request validation và response model
security/	SecurityFilterChain, JWT encoder/decoder, current user
frontend/src/pages	Trang public, user và admin
frontend/src/services	Axios và API client tập trung
frontend/src/context	AuthContext và ThemeContext

4.2. Backend

4.2.1. Story và tìm kiếm

StoryRepository sử dụng JPQL với các filter tùy chọn: keyword, status, genreSlug và visibility. Query public luôn ép visibility=PUBLIC; query admin cho phép xem cả nội dung ẩn. Genre join có thể nhân bản row nên query dùng distinct và countQuery riêng. Sort key từ client được ánh xạ theo danh sách cho phép thay vì đưa tên cột tùy ý vào truy vấn.

4.2.2. Chapter và điều hướng

Chapter có unique constraint (story_id, chapter_number). Điều hướng dùng truy vấn nearest-neighbor để bỏ qua khoảng trống hoặc chương ẩn. Khi đổi thứ tự, service tạm đưa một chương về số âm, flush, hoán đổi rồi flush lại nhằm tránh vi phạm unique constraint ở trạng thái trung gian.

4.2.3. Rating và dữ liệu tổng hợp

Mỗi user có tối đa một rating cho một story. ratingAvg và ratingCount được lưu denormalized trong Story để sắp xếp danh sách hiệu quả. Sau khi tạo, cập nhật hoặc xóa rating, service tính lại aggregate và bulk update Story mà không kích hoạt updatedAt.

4.2.4. Xóa dữ liệu

Story không khai báo cascade mở hồ. StoryService xóa explicit các comment report, comment, rating, reading progress, follow và chapter theo thứ tự phụ thuộc trong một transaction. Cách làm này khiến chính sách xóa có thể đọc và kiểm soát được, đồng thời tránh lỗi foreign key.

4.3. Frontend

React Router chia route public, route cần đăng nhập và route ADMIN. RequireAuth/RequireAdmin giúp điều hướng UX, nhưng backend vẫn là ranh giới bảo mật. Mọi API call nằm trong services; component không tự tạo Axios request rải rác. AuthContext quản lý phiên; ThemeContext quản lý giao diện sáng/tối.

Trang đọc chương lưu cỡ chữ và giãn dòng trong localStorage; theme dùng context toàn cục. Tiến độ đọc được ghi nền cho user nhưng lỗi lưu bookmark không làm gián đoạn việc đọc. Các trang danh sách đặt filter trong URL để có thể chia sẻ và giữ trạng thái sau khi tải lại.

4.4. Các chức năng quản trị

- CRUD truyện: title, slug, author, description, cover, status, visibility, nhiều genre.
- CRUD chương: content, publish/unpublish, reorder, xóa an toàn với bookmark.
- CRUD thể loại: name, slug, description; không xóa nếu đang được story sử dụng.
- Quản lý user: tìm kiếm, khóa/mở khóa, nâng/hạ role; chặn tự thay đổi chính mình.
- Moderation: tìm toàn bộ comment; lọc reported/hidden; ẩn/khôi phục/xóa; khóa tác giả.
- Dashboard: tổng user/story/chapter/comment, nội dung cập nhật 7 ngày, top view/follow.

CHƯƠNG 5. KIỂM THỬ, TRIỂN KHAI VÀ KẾT QUẢ

5.1. Chiến lược kiểm thử

Backend test dùng H2 in-memory ở MySQL compatibility mode, vì vậy bộ test có thể chạy trên CI hoặc máy mới mà không cần MySQL/Docker. MockMvc kiểm tra cả HTTP status và response JSON. Spring Security Test tạo ngữ cảnh USER/ADMIN để kiểm tra ma trận phân quyền.

Nhóm	Kịch bản tiêu biểu	Kết quả
Context	Spring context khởi động thành công	Đạt
Visibility	Danh sách public bỏ PRIVATE; detail PRIVATE trả 404	Đạt
Chapter	Ẩn draft; prev/next; tăng view không đổi updatedAt	Đạt
Search	Khớp author và genre	Đạt
Security	Anonymous 401; USER 403; ADMIN truy cập được	Đạt
Auth	Login bằng email trả JWT	Đạt
Moderation	Comment hidden biến mất khỏi API public	Đạt
Reorder	Move up/down, edge no-op, số chương vẫn unique	Đạt

Kết quả hiện tại: 17 kiểm thử backend chạy thành công, không có failure hoặc error. Frontend production build thành công với 135 module được biến đổi. Postman collection hợp lệ và được cập nhật cho API moderation, genre description và login.

5.2. Triển khai

Thành phần	Nền tảng	Cấu hình chính
Frontend	Vercel	Root frontend/, VITE_API_BASE_URL, SPA fallback
Backend	Render	Dockerfile, PORT, DB_URL, JWT_SECRET, CORS
Database	Aiven MySQL	JDBC SSL, tài khoản qua environment variables

Backend trên gói miễn phí có thể sleep sau thời gian không truy cập, khiến lần gọi đầu mất thêm thời gian. Đây là giới hạn hạ tầng chứ không phải lỗi nghiệp vụ. Cấu hình production được tách khỏi mã nguồn thông qua biến môi trường; JWT secret, database password và admin password không được commit.

5.3. Đánh giá kết quả

Sản phẩm hoàn thành các yêu cầu F01–F22 theo phạm vi đề tài: từ đọc truyện công khai đến thư viện cá nhân và quản trị đầy đủ. Điểm mạnh là kiến trúc rõ ràng, bảo mật được enforce ở backend, dữ liệu có constraint, API phân trang, giao diện responsive và có kiểm thử tự động. Sản phẩm đã vượt mức CRUD minh họa để trở thành một ứng dụng end-to-end có thể triển khai.

CHƯƠNG 6. KINH NGHIỆM THỰC TẬP VÀ NHỮNG ĐIỀU HỌC ĐƯỢC

6.1. Khác biệt với việc học làm web trên trường

Ở trường, bài tập thường có đề bài ổn định, dữ liệu mẫu nhỏ và tiêu chí chấm tập trung vào kết quả đúng. Các phần frontend, backend hoặc cơ sở dữ liệu có thể được học riêng. Trong dự án thực tập, một thay đổi nhỏ ở giao diện có thể kéo theo contract API, validation, quyền truy cập, schema và test. Vì vậy, kỹ năng quan trọng không chỉ là biết cú pháp mà là nhìn được luồng xuyên suốt hệ thống.

Khía cạnh	Trong học phần	Trong dự án thực tế
Yêu cầu	Đề bài rõ, ít thay đổi	Phạm vi tăng dần; cần ưu tiên MVP và giữ tương thích
Dữ liệu	Có thể tạo lại	Có dữ liệu cũ; thêm cột cần default/migration
Bảo mật	Thường minh họa login	401/403, role, ownership, secret, CORS là ranh giới thật
Lỗi	Lỗi trong một bài	Có thể nằm ở UI, network, security filter, service, ORM hoặc cloud
Chất lượng	Chạy đúng happy path	Validation, null, duplicate, not found, transaction, test
Triển khai	Chạy localhost	Docker, biến môi trường, CORS domain, cold start, SSL
Làm việc	Nộp một phiên bản	Git history, review diff, docs, API contract, khả năng bảo trì

6.2. Bài học kỹ thuật cụ thể

Kiến trúc phân lớp

Việc tách controller, service và repository không phải hình thức. Khi logic dồn vào controller, khó kiểm thử và khó tái sử dụng. MangaVault đặt nghiệp vụ ở service và giữ controller mỏng.

DTO và API contract

Entity phản ánh database, còn DTO phản ánh dữ liệu client được phép gửi/nhận. Tách hai khái niệm giúp validation rõ và ngăn lộ trường nhạy cảm.

JWT và Spring Security

401/403 xảy ra trong filter chain nên không đi qua GlobalExceptionHandler. Muốn response thống nhất phải cấu hình AuthenticationEntryPoint và AccessDeniedHandler. HS256 cũng cần khai báo đúng JWS header cho encoder.

CORS

CORS không chỉ là thêm header. Preflight phải đi qua SecurityFilterChain; vì vậy cấu hình CorsConfigurationSource ở đúng tầng quan trọng hơn việc thêm cấu hình rồi ở MVC.

Transaction

Xóa story liên quan nhiều bảng và đổi thứ tự chapter đều có trạng thái trung gian. Transaction bảo đảm hoặc hoàn thành toàn bộ hoặc rollback, tránh dữ liệu dở dang.

ORM và hiệu năng

Lazy loading tránh tải thừa nhưng có thể gây N+1. EntityGraph, batch fetch và truy vấn theo nhóm được dùng ở comment/follow để giảm số query.

Dữ liệu tổng hợp

View count và rating aggregate được cập nhật bằng bulk query để không làm thay đổi updatedAt, bởi thao tác đọc/đánh giá không đồng nghĩa nội dung truyện vừa cập nhật.

Kiểm thử

Test có giá trị nhất ở những ranh giới dễ hỏng: visibility, authorization, unique reorder, counter và hidden comment. H2 giúp chạy nhanh nhưng vẫn cần ý thức về khác biệt với MySQL.

Frontend state

State trong React có thể cũ trong closure. Filter được đặt trong URL để tránh mất trạng thái; auth storage được đồng bộ lại khi interceptor xóa token.

Deployment

Một ứng dụng chạy local chưa chứng minh rằng cấu hình CORS, URL API, SSL database, PORT và secret đúng trên cloud. Deployment là một phần của phát triển, không phải bước phụ cuối cùng.

6.3. Những vấn đề thực tế đã gặp và cách giải quyết

Vấn đề	Nguyên nhân	Cách xử lý
JWT encoder không chọn được khóa ký	Thiếu JWS header HS256	Khai báo MacAlgorithm.HS256 khi encode
Thêm chương không cập nhật story	@UpdateTimestamp chỉ chạy khi entity story dirty	Chủ động cập nhật updatedAt trong transaction
Reorder vi phạm unique	Hai chapter tạm thời có cùng chapterNumber	Dùng số âm tạm và saveAndFlush theo bước
Điều hướng sai khi thiếu chương	Giả định chapterNumber ± 1	Truy vấn chương gần nhất nhỏ hơn/lớn hơn
UTF-8 tiếng Việt hỏng trên Windows	Cách truyền dữ liệu inline qua shell	Dùng file UTF-8 và data-binary
Frontend gọi API production sai	Biến Vite được nhúng lúc build	Cấu hình VITE_API_BASE_URL và redeploy

6.4. Kỹ năng làm việc và tư duy nghề nghiệp

- Biết chia phạm vi thành MVP và các giai đoạn thay vì phát triển đồng thời mọi chức năng.
- Đọc lỗi theo tầng: browser → network → security → controller → service → repository → database.
- Giữ thay đổi nhỏ, kiểm tra diff và không tự ý sửa file ngoài phạm vi.
- Viết thông điệp lỗi và API contract từ góc nhìn người sử dụng frontend.
- Không xem deployment, test và tài liệu là phần việc có thể bỏ qua.
- Ưu tiên code dễ đọc và quyết định nghiệp vụ rõ ràng hơn kỹ thuật trừu tượng quá mức.

6.5. Tự đánh giá sự tiến bộ

Trước khi thực hiện dự án, em chủ yếu nhìn một chức năng dưới dạng form–controller–database. Sau quá trình thực tập, em đã có thể phân tích thêm ranh giới quyền, ownership, trạng thái lỗi, dữ liệu cũ, hiệu năng query, tác động triển khai và khả năng kiểm thử. Sự thay đổi lớn nhất là chuyển từ tư duy “viết code để chạy” sang “xây hệ thống có thể giải thích, kiểm tra, triển khai và bảo trì”.

CHƯƠNG 7. HẠN CHẾ VÀ HƯỚNG PHÁT TRIỂN

7.1. Hạn chế hiện tại

- JWT stateless chưa có refresh token/revocation; thay đổi role cần chiến lược làm mất hiệu lực token rõ ràng.
- Ảnh bìa dùng URL; chưa có upload, resize và CDN do hệ thống quản lý.
- Chưa có rate limiting, chống brute-force và quan sát hệ thống chuyên sâu.
- Chưa hỗ trợ tác giả tự đăng truyện, thông báo chương mới hoặc đề xuất cá nhân hóa.

7.2. Hướng phát triển

Giai đoạn	Nội dung ưu tiên
Ngắn hạn	Flyway migration; tắt open-in-view; test auth/library/comment/rating; chuẩn hóa validation pagination
Trung hạn	Refresh token/revocation; upload ảnh; rate limiting; logging/monitoring; CI GitHub Actions
Dài hạn	Tác giả đăng nội dung; thông báo chương; tìm kiếm toàn văn; đề xuất cá nhân hóa; cache/CDN

KẾT LUẬN

Đề tài MangaVault đã xây dựng thành công website đọc truyện trực tuyến theo kiến trúc React–Spring Boot–MySQL. Sản phẩm hỗ trợ đầy đủ ba nhóm người dùng, hoàn thành các chức năng từ khám phá và đọc truyện đến thư viện cá nhân, tương tác và quản trị. Hệ thống sử dụng JWT, BCrypt, DTO, validation, exception handling, transaction, pagination và các biện pháp tối ưu truy vấn phù hợp với phạm vi.

Quan trọng hơn kết quả chức năng là quá trình chuyển đổi tư duy. Qua thực tập, em hiểu rằng phần mềm thực tế không kết thúc ở CRUD hay giao diện chạy được. Chất lượng hình thành từ nhiều quyết định nhỏ: xử lý dữ liệu không tồn tại, bảo vệ quyền sở hữu, tránh N+1, không làm sai updatedAt, giữ tương thích API, cấu hình môi trường và viết test cho ranh giới dễ hỏng. Những bài học này khác biệt rõ với bài tập web trong môi trường học tập được kiểm soát và là hành trang trực tiếp cho công việc phát triển phần mềm sau khi tốt nghiệp.